

SERVER SIDE MODULE

CONTENTS

This module has the following files:

1. SERVER MODULE.docx
2. SERVER MODULE.pdf
3. SERVER MODULE MEDIA.zip

INTRODUCTION

The client has requested the development of a **backend system** and **frontend web application** for a **Website Builder Platform** called **WelBuilder**. The system allows users to create and manage web pages using predefined section templates.

Users can register and log in to the application. After logging in, users are able to create pages and organize them by adding multiple sections. Each section is based on a template that defines its structure, and contains fields such as text or image that can be filled by the user.

This platform is designed to help users build web pages easily without writing code. By using templates and dynamic content, users can create structured and consistent pages in a simple way.

PROJECT OVERVIEW

This project consists of two main phases. In Phase 1, you are required to develop a REST API using the Laravel Framework. The API must implement core features such as user authentication, page management, section management, and dynamic content handling based on templates.

In Phase 2, you will build a frontend web application that integrates with the API. You may use either React or Vue as your framework of choice.

The system allows users to create pages and build their structure by adding sections. Each section is based on a predefined template, which contains multiple fields. Users can fill these fields with content such as text or images.

To support your work, media files have been provided, including Environment, a Database Dump (.sql) and Framework Scaffolding.

PHASE 1 - Backend Development (REST API)

A. Authentication

There are three endpoints: register, login, and logout. **Sanctum** should be used to create tokens. The generated token will be used to access all available endpoints. The logout function should only deactivate the token from the specific device making the request.

A1: Register

POST /api/register

For users to create an account in the application.

Request Headers

Accept	application/json
--------	------------------

Body (JSON)

name	required, string.
email	required, valid email format, unique.
password	required, min 6 chars.

A1a: Response Success

Status Code	Body
201	<pre>{ "status": "success", "message": "Registration successful", "data": { "id": 1, "name": "Budi", "email": "budi@visualcraft.id", "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T16:01:46.000000Z", } }</pre>

	<pre> "token": "1 wwu0rgoVe4Xyj5dvUmnf..." } } </pre>
--	---

Response Invalid Field

Status Code	Body
422	<pre> { "status": "error", "message": "Invalid field", "errors": { ... } } </pre>

A2: Login

POST /api/login

For users to log in to the application using email and password.

Request Headers

Accept	application/json
--------	------------------

Body (JSON)

email	required.
password	required.

A2a: Response Success

Status Code	Body
200	<pre> { "status": "success", "message": "Login successful", "data": { "id": 1, "name": "Budi", "email": "budi@visualcraft.id", "created_at": "2025-07-11T16:01:38.000000Z", "updated_at": "2025-07-11T16:01:38.000000Z", "token": "2 0Q2TAdua0F0slp1F4cx..." } } </pre>

A2b: Response Failed

Status Code	Body
401	<pre>{ "status": "error", "message": "Username or password incorrect" }</pre>

A3: Logout

POST /api/logout

For logged-in users to log out of the application. It specifically deactivates the authentication token associated with the current device making the request.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

A3a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Logout successful" }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

B. Page Management

The application allows authenticated users to manage their **Pages**. Each page has a unique **slug**, a **title**, and an optional **summary**. Pages belong to the authenticated user and can only be managed by their owner.

B1: Create Page

POST /api/pages

For logged-in users to create a new page.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

title	required, string.
slug	required, string, unique, only lowercase letters, numbers, and hyphens.
summary	optional, string.

B1a: Response Success

Status Code	Body
201	<pre>{ "status": "success", "message": "Page created successful", "data": { "id": 1, "user_id": 1, "title": "My First Page", "slug": "my-first-page", "summary": "A short description of this page.", "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T16:01:46.000000Z" } }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error",</pre>

	<pre> "message": "Unauthenticated." } </pre>
--	--

Response Invalid Field

Status Code	Body
422	<pre> { "status": "error", "message": "Invalid field", "errors": { ... } } </pre>

B2: Get All Pages

GET /api/pages

For logged-in users to get all of their own pages.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

B2a: Response Success

Status Code	Body
200	<pre> { "status": "success", "message": "Get all pages successful", "data": { "pages": [{ "id": 1, "user_id": 1, "title": "My First Page", "slug": "my-first-page", "summary": "A short description.", "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T16:01:46.000000Z" }, ...] } } </pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

B3: Get Page by ID

GET /api/pages/:id

For logged-in users to get a specific page by its slug, including all composed sections and their field values.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

B3a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Get page successful", "data": { "id": 1, "user_id": 1, "title": "My First Page", "slug": "my-first-page", "summary": "A short description.", "sections": [{ "id": 1, "position": 1, "template": { "id": 1, "name": "Hero Banner", "slug": "hero-banner" } }, { "id": 1, "position": 2, "template": { "id": 1, "name": "Text", "slug": "text" }, "fields": [{ "id": 1, "name": "Heading", "slug": "heading", "type": "text", "value": "Welcome to VisualCraft" }] }] } }</pre>

	<pre> ...] }, ...], "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T16:01:46.000000Z" } } </pre>
--	---

Response Invalid Token

Status Code	Body
401	<pre> { "status": "error", "message": "Unauthenticated." } </pre>

Response Forbidden

Status Code	Body
403	<pre> { "status": "error", "message": "Forbidden access" } </pre>

Response Not Found

Status Code	Body
404	<pre> { "status": "error", "message": "Not found" } </pre>

B4: Update Page

PUT /api/pages/:slug

For logged-in users to update their own page.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

title	optional, string.
slug	optional, string, unique, only lowercase letters, numbers, and hyphens.
summary	optional, string.

B4a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Page updated successful", "data": { "id": 1, "user_id": 1, "title": "Updated Page Title", "slug": "updated-page-title", "summary": "Updated summary.", "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T17:00:00.000000Z" } }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

Response Forbidden

Status Code	Body
403	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

Response Not Found

Status Code	Body
404	<pre>{ "status": "error", "message": "Not found" }</pre>

Response Invalid Field

Status Code	Body
422	<pre>{ "status": "error", "message": "Invalid field", "errors": { ... } }</pre>

B5: Delete Page

DELETE /api/pages/:slug

For logged-in users to delete their own page. Deleting a page also removes all associated page sections and field values.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

B5a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Page deleted successful" }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

Response Forbidden

Status Code	Body
403	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

	}
--	---

Response Not Found

Status Code	Body
404	<pre>{ "status": "error", "message": "Not found" }</pre>

C. Template Management

Templates define the structure of a reusable content block. Each template has a **name**, a unique **slug**, and one or more **Template Fields**. Each field has a **name**, a **slug**, and a **type** which can be either **text** or **image**. Section Templates are global and accessible to all authenticated users.

There are some provided templates that you should create.

ID	Template Name	Slug	Fields
1	Hero Section	hero	title, subtitle, background_image
2	About Section	about	heading, description, image
3	Services Section	services	service_1_name service_1_icon service_2_name service_2_icon service_3_name service_3_icon
4	Contact Section	contact	title, email, phone, map_image
5	Footer Section	footer	copyright_text, logo_url, instagram_link, facebook_link

tC1: Get All Templates

GET /api/templates

For logged-in users to get all available section templates including their fields.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

C1a: Response Success

Status Code	Body
200	{ "status": "success", "message": "Get all templates successful", }

	<pre> "data": { "templates": [{ "id": 1, "name": "Hero Banner", "slug": "hero-banner", "fields": [{ "id": 1, "template_id": 1, "name": "Heading", "slug": "heading", "type": "text" }, { "id": 2, "template_id": 1, "name": "Background Image", "slug": "background-image", "type": "image" }, ...] }, ...] } </pre>
--	--

Response Invalid Token

Status Code	Body
401	<pre> { "status": "error", "message": "Unauthenticated." } </pre>

C2: Get Template by Slug

GET /api/templates/:slug

For logged-in users to get a specific template including all of its fields.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

C2a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Get template successful", "data": { "id": 1, "name": "Hero Banner", "slug": "hero-banner", "fields": [{ "id": 1, "template_id": 1, "name": "Heading", "slug": "heading", "type": "text" }, ...] } }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

Response Not Found

Status Code	Body
404	<pre>{ "status": "error", "message": "Not found" }</pre>

D. Page Section Management

Page Sections compose a page by linking a **Page** with a **Section Template** at a specific **position**. Each section may have field values that provide the actual content for the template fields. Sections can be reordered, added, or removed from a page. Only the page owner can manage its sections.

D1: Add Section to Page

POST /api/pages/:slug/sections

For logged-in users to add a new section to their page using a section template.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

template_id	required, integer, must be a valid section template ID.
position	required, integer, starting from 1.

D1a: Response Success

Status Code	Body
201	<pre>{ "status": "success", "message": "Section added successful", "data": { "id": 1, "page_id": 1, "template_id": 1, "position": 1, "template": { "id": 1, "name": "Hero Banner", "slug": "hero-banner" }, "fields": [{ "id": 1, "name": "Heading", "slug": "heading", "type": "text", "value": null },], } }</pre>

	<pre> ...], "created_at": "2025-07-11T16:01:46.000000Z", "updated_at": "2025-07-11T16:01:46.000000Z" } } </pre>
--	--

Response Invalid Token

Status Code	Body
401	<pre> { "status": "error", "message": "Unauthenticated." } </pre>

Response Forbidden

Status Code	Body
403	<pre> { "status": "error", "message": "Forbidden access" } </pre>

Response Not Found

Status Code	Body
404	<pre> { "status": "error", "message": "Not found" } </pre>

Response Invalid Field

Status Code	Body
422	<pre> { "status": "error", "message": "Invalid field", "errors": { ... } } </pre>

D2: Update Section Field Values

PUT /api/pages/:slug/sections/:sectionId/fields

For logged-in users to update the field values of a specific section on their page. The request body should contain an array of field values.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

fields	required, array of objects: { field_id: integer, value: string }.
--------	---

D2a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Section fields updated successful", "data": { "id": 1, "page_id": 1, "template_id": 1, "position": 1, "fields": [{ "id": 1, "name": "Heading", "slug": "heading", "type": "text", "value": "Welcome to VisualCraft" }, ...] } }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

Response Forbidden

Status Code	Body
403	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

Response Not Found

Status Code	Body
404	<pre>{ "status": "error", "message": "Not found" }</pre>

Response Invalid Field

Status Code	Body
422	<pre>{ "status": "error", "message": "Invalid field", "errors": { ... } }</pre>

D3: Reorder Sections

PUT /api/pages/:slug/sections/reorder

For logged-in users to reorder the sections on their page. The request body should contain an ordered array of section IDs.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

sections	required, array of integers (section IDs in desired order).
----------	---

D3a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Sections reordered successful" }</pre>

	}
--	---

Response Invalid Token

Status Code	Body
401	{ "status": "error", "message": "Unauthenticated." }

Response Forbidden

Status Code	Body
403	{ "status": "error", "message": "Forbidden access" }

Response Not Found

Status Code	Body
404	{ "status": "error", "message": "Not found" }

Response Invalid Field

Status Code	Body
422	{ "status": "error", "message": "Invalid field", "errors": { ... } }

D4: Remove Section from Page

DELETE /api/pages/:slug/sections/:sectionId

For logged-in users to remove a section from their page. This also deletes all associated field values.

Request Headers

Accept	application/json
---------------	------------------

Authorization	Bearer <TOKEN>
---------------	----------------

D4a: Response Success

Status Code	Body
200	<pre>{ "status": "success", "message": "Section removed successful" }</pre>

Response Invalid Token

Status Code	Body
401	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

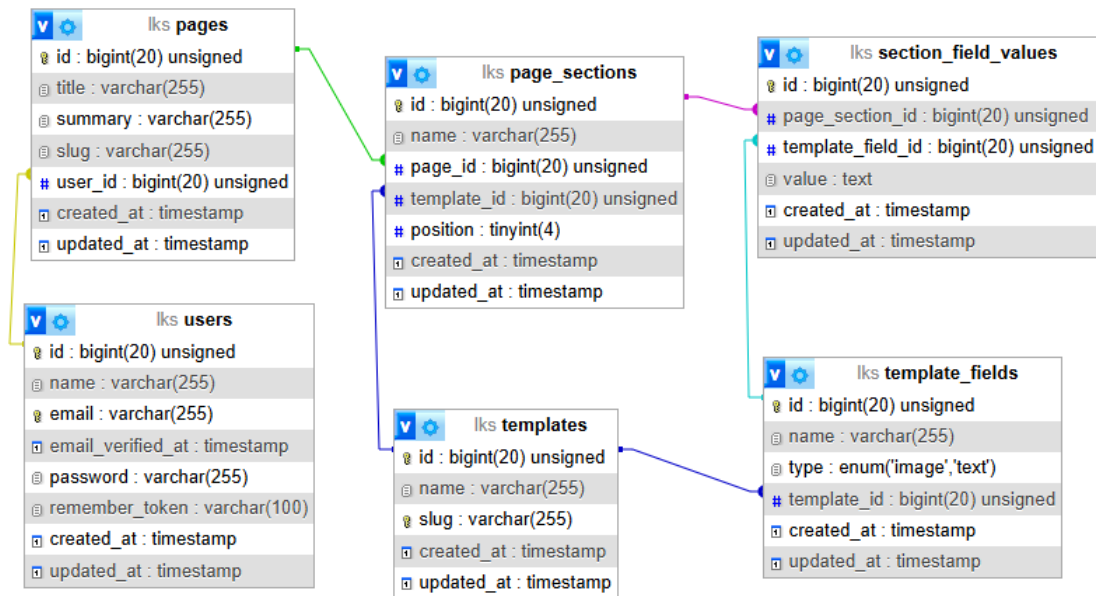
Response Forbidden

Status Code	Body
403	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

Response Not Found

Status Code	Body
404	<pre>{ "status": "error", "message": "Not found" }</pre>

ER DIAGRAM



PHASE 2 - Frontend Web Application

Pages	Features
General	• The document title should reflect the current page (e.g., "Login", "Register", "Pages", "Page Builder"). • After a successful login, display a Logout button in the navbar. • Users can log out by clicking the Logout button in the navbar.
Register	Path: /register • Users can register using the following fields: ◦ Name ◦ Email ◦ Password • Display specific error messages based on the relevant input field when registration fails. • After a successful registration, users are redirected to the Pages page. • If a logged-in user accesses this page, they will be redirected to the Pages page automatically.
Login	Path: /login • Users can log in using their existing credentials (email and password). • Display specific error messages when login fails, based on the relevant input field. • After a successful login, users are redirected to the Pages page. • If a logged-in user accesses this page, they will be redirected to the Pages page automatically.
Pages	Path: /pages Access: Only accessible to logged-in users Displayed Information • List of all pages owned by the logged-in user. • Each page item shows: ◦ Page title

	◦ Page slug ◦ Page summary (if provided) Actions • Create Page ◦ Click the "+ New Page" button. ◦ Fill in the title, slug, and optional summary, then click "Save". ◦ The new page appears in the list. • Edit Page ◦ Click the "Edit" button on a page item. ◦ Update the fields and click "Save". • Delete Page ◦ Click the "Delete" button on a page item. ◦ Confirm the deletion in the popup dialog and click "Ok" to delete. • Open Page Builder ◦ Click the page title or a "Build" button to navigate to the Page Builder.
Page Builder	Path: /pages/:slug/builder Access: Only accessible to logged-in users Navigation • Users access this page by clicking a page item from the Pages list. Displayed Information • Page Information ◦ Display the page title and slug. • Section List ◦ Show all sections added to the page, in order. ◦ Each section shows its template name and the current field values. Actions • Add Section ◦ Click the "Add Section" button.

	◦ Choose a section template from a dropdown list. ◦ The section is added at the bottom of the page. • Edit Section Fields ◦ Click on a section to expand the field editor. ◦ Fill in text fields or provide image URLs for image fields. ◦ Click "Save" to persist the field values. • Reorder Sections ◦ Drag and drop sections to reorder them. ◦ The new order is saved automatically. • Remove Section ◦ Click the "Remove" button on a section item. ◦ Confirm the removal in the popup dialog. • Preview Page ◦ Click the "Preview" button to see a live preview of the page in its current state. ◦ The preview renders each section using its template and field values.
Page Preview	Path: /pages/:slug/preview Access: Only accessible to logged-in users Displayed Information • Render all sections in order with their actual content. • Text fields display their saved text value. • Image fields display the image from the provided URL. Navigation • Include a "Back to Builder" button to return to the Page Builder.

Shortcuts

Keymap	Description
Alt + N	Open "New Page" form

Alt + S	Open "Add Section" panel
Esc	Close the form or panel

INSTRUCTION FOR COMPETITORS

1. You can use the provided HTML template to build your frontend. You may also modify it or add additional elements and pages as needed.
2. You can add extra fields to the response body if needed, **just make sure all required fields are included and the response structure is correct.**
3. You are allowed to create some api-endpoints for the frontend's needs.
4. Create a root folder called **XX_MODULE_SERVER** in your local computer, where **XX** is your computer number.
5. Export **db-dump.sql** and **db-diagram.pdf** to the root **XX_MODULE_SERVER** folder.
6. In the **XX_MODULE_SERVER** folder, create two folders:
 1. backend/ which is laravel project
 2. frontend/ which is react/vue project
 3. database/ which contains db-dump.sql and db-diagram.pdf
7. Zip your root folder **XX_MODULE_SERVER** and submit to the submission page.
8. Exclude the following folders when zipping:
 - vendor/
 - node_modules/